

Correlation-Adaptive LSTM for Multi-factor Stock Return Forecasting

WONG Yan Kit Xinyu Mao ZHU Zherui
Under the supervision of Prof. CHOU Hongsong, HKUST

May 2025

Abstract

This study proposes a novel Correlation-Adaptive Long Short-Term Memory (LSTM) framework to enhance multi-factor stock return forecasting by incorporating inter-stock correlations across three time frequencies: monthly, 5-minute, and 1-minute. There are two primary strategies: modifying input data structures and adapting the LSTM architecture. At the monthly level, we implement clustering and CNN-LSTM with spatial attention. Hierarchical clustering with nine clusters achieves the best performance and it is significantly outperforming the baseline Simple LSTM. At the 5-minute level, a hybrid CNN-LSTM model with multi-resolution processing and attention mechanisms is applied, which also reduces the MSE and achieves a better IC. At the minute level, a Correlation Memory Control Ensemble LSTM reduces MSE and increases IC compared to an independent model, with leading stocks benefiting most. These findings demonstrate that incorporating inter-stock correlations enhances prediction accuracy across diverse time scales, offering a robust framework for financial forecasting in both low- and high-frequency environments.

1 Introduction

Stock return forecasting remains a critical challenge in quantitative finance, particularly in capturing the complex interplay of temporal dynamics and cross-sectional relationships among assets. Inter-stock correlations can provide valuable signals for improving predictive accuracy. Long Short-Term Memory (LSTM) networks, designed to model long-term dependencies in sequential data, are well-suited for financial time-series forecasting. This study proposes a Correlation-Adaptive LSTM framework to integrate these correlations into stock return forecasting across three distinct time frequencies: monthly, 5-minute and 1-minute.

Performance is evaluated using Mean Squared Error (MSE) and Information Coefficient (IC), with a Simple LSTM serving as the baseline. We hypothesize that incorporating inter-stock correlations will enhance prediction accuracy across all frequencies, with tailored approaches addressing the unique challenges of each time scale. For monthly data, we expect clustering and architectural modifications to improve both MSE and IC. For 5-minute data, the CNN-LSTM model aims to capture subtle cross-sectional patterns in a noisy environment. For minute-level data, the ensemble approach seeks to balance bias and variance to improve robustness. This study aims to provide a comprehensive framework for correlation-adaptive forecasting, offering insights into its applicability across diverse financial contexts.

2 Methodology

2.1 Monthly Level

Data This study uses monthly data spanning from January 23, 2003, to February 27, 2025, covering 112 stocks from the Taiwan market. In the monthly dataset, each stock is characterized by 55 features, including technical indicators computed at various frequencies (e.g., RSI over 5-day and 20-day windows; SMA over 5-day and 20-day windows) as well as WorldQuant Alpha factors. The latter are derived from the publicly available framework of WorldQuant’s 101 Alpha formulas (e.g., ‘WQ_Alpha_05_5days’, ‘WQ_Alpha_05_20days’, ‘WQ_Alpha_05_60days’). These indicators span multiple time horizons—such as 5-day, 20-day, and 60-day windows—for the same feature, enabling the model to capture market signals across different frequencies. The prediction target is the monthly stock return. There exists substantial correlation both among stocks and among factors. The primary aim of this study is to investigate how inter-stock relationships and shared factor structures influence the prediction of stock returns.

The research is based on LSTM models and primarily explores two categories of approaches to incorporate inter-stock correlations into the LSTM framework, resulting in a total of four different model implementations.

When incorporating inter-stock correlation into the model, two main approaches are considered. The first approach keeps the LSTM architecture largely unchanged while modifying the structure of the input data to reflect stock correlations. The second approach involves altering the LSTM architecture itself—for example, by adjusting the input gate or introducing a CNN layer—to separate the processing of correlation information from temporal dynamics, thereby embedding stock correlation directly into the model design.

2.1.1 Simple LSTM

Long Short-Term Memory (LSTM) networks are a specialized type of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data. The core of the Long Short-Term Memory (LSTM) model lies in its gating mechanisms, which are responsible for updating the hidden state $\mathbf{h}_t$ and the cell state $\mathbf{c}_t$. These gates allow the model to learn long-term dependencies by controlling the flow of information through the cell.

Forget Gate The forget gate determines which information from the previous cell state should be discarded:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (1)$$

This gate controls the extent to which information from the past is retained or removed from the memory.

Input Gate The input gate regulates how much new information should be added to the cell state:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (2)$$

Together with the candidate memory content, it determines which parts of the new input are relevant for updating the cell state.

Candidate Memory Content The candidate values $\tilde{\mathbf{c}}_t$ represent new information that could be added to the cell state:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (3)$$

Cell State Update The new cell state $\mathbf{c}_t$ is computed by combining the old state and the new candidate information:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (4)$$

Output Gate and Hidden State Finally, the output gate determines what to output from the current cell state:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (5)$$

The hidden state $\mathbf{h}_t$ is then computed as:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (6)$$

Before adjusting the model to incorporate correlation, we first use a simple LSTM model without considering inter-stock correlation as the baseline. The subsequent models will be compared against this benchmark, primarily in terms of Mean Squared Error (MSE). For models that achieve lower MSE, we additionally report their Information Coefficient (IC) as a supplementary evaluation metric.

In the following part, we will introduce two methods to change the structure of the input data.

Change the structure of the input data:

2.1.2 LSTM with Clustering

To incorporate stock correlations, we first adopt an intuitive approach by clustering stocks. The first method applies KMeans clustering based on historical returns to group stocks, and within each cluster, an LSTM model with shared weights is trained to predict stock returns. When training the LSTM for each individual stock within a cluster, weight inheritance is applied, while models across different clusters remain independent.

The second clustering method constructs a distance matrix based on stock correlations, defined as $1 - \text{corr}$, and performs hierarchical clustering. The handling of LSTM model weights—shared within clusters and independent across clusters—follows the same approach as in the KMeans-based method.

2.1.3 LSTM with Feature Concatenation

In addition to clustering-based approaches, this study also explores feature concatenation to incorporate inter-stock correlation. For each individual stock, features from highly correlated stocks are concatenated with its own features and fed into the LSTM model. This method aims to assess how inter-stock correlation affects prediction accuracy.

Specifically, the process begins by calculating the pairwise correlations among all stocks in the market. For a given target stock—the stock we aim to predict—we identify the top k most correlated stocks as its neighboring stocks. If a neighboring stock shares less than 70% of overlapping time periods with the target stock, it is excluded and replaced by the next most correlated stock, until k valid neighbors are selected. The features of these neighboring stocks are then horizontally concatenated with those of the target stock to form a wide input table, which is subsequently used to train the LSTM model.

In the following part, we will introduce two methods to change the structure of the model.

Change the structure of the model:

2.1.4 LSTM with Correlation

As discussed earlier, the core of the LSTM model lies in its gating mechanism. The key innovation in this method is the integration of cross-stock correlation information directly into the input gate of the LSTM. This modification aims to explicitly incorporate inter-stock correlation into the model and evaluate its impact on prediction accuracy.

To achieve this, we introduce contextual information from correlated stocks into the input gate. The modified input gate equation is defined as follows:

$$\mathbf{i}_t = \sigma \left(\underbrace{\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t]}_{\text{basic term}} + \underbrace{\mathbf{W}_{c_extra} \cdot \text{context}_t}_{\text{context term}} + \mathbf{b}_i \right) \quad (7)$$

where:

- **context_t**: The context vector at time t , dynamically constructed using a predefined correlation matrix (denoted as `alpha_matrix`). It captures relevant information from correlated stocks.
- **W_{c_extra}**: An additional weight matrix used to project the contextual information into the input gate computation.

This formulation allows the input gate to account not only for the temporal dynamics of the target stock, but also for cross-sectional signals from related stocks in the market.

For the construction of the context vector context_t , first, a stock-to-stock correlation matrix—referred to as the **alpha matrix**, is defined for the entire market. For a given target stock, we identify its correlated stocks (neighbors) from the alpha matrix. For each neighboring stock $j \in \text{neighbors}$, we extract its hidden state at time $t - 1$. The context vector is then constructed using the following formulation:

$$\text{context}_t[:, i] = \sum_{j \in \text{neighbors}} \alpha_matrix[i, j] \cdot \text{mean} \mathbf{h}_{t-1}^{(j)} \quad (8)$$

Here, $\text{mean}(\mathbf{h}_{t-1}^{(j)})$ denotes the average hidden state of stock j at time $t-1$, and $\text{alpha_matrix}[i, j]$ is the correlation weight between stock i and stock j . This formulation ensures that contextual information from correlated stocks is integrated into the prediction for stock i .

Compared to the standard LSTM, the correlation-enhanced LSTM introduces an additional weight parameter, $\mathbf{W}_{c_extra}$, which, like the other LSTM parameters, is optimized by back-propagation. Highly correlated stocks are selected based on the predefined alpha_matrix , and their hidden states are aggregated to form a context vector. The input gate then adjusts its information integration strategy according to this contextual input. By projecting the context vector into the input gate through the $\mathbf{W}_{c_extra}$ matrix, the model is able to nonlinearly fuse information from multiple sources, enabling it to adapt to complex inter-stock dependency patterns.

2.1.5 CNN-LSTM

The second method is to add CNN layers before LSTM to construct CNN-LSTM model.

CNN’s capability to extract spatial features enables the model to capture both local and global correlations among stocks. By sliding convolutional kernels across a structured stock grid, CNN layers learn patterns of inter-stock relationships. Incorporating CNN prior to LSTM allows for the separation of spatial and temporal modeling: the CNN handles cross-sectional correlations among stocks, while the LSTM focuses on temporal dependencies.

An essential design consideration in the CNN component is how to structure the stock grid. In this study, we adopt the Market Cube framework, where the stock grid is organized in a $\text{stock} \times \text{stock} \times \text{features}$ format. For example, in the case of monthly data with 112 stocks, we reshape the market into a 14×8 2D image. The depth dimension of this image represents the feature channels for each stock—similar to how RGB channels are used in image processing. Here, the 55 features are treated as channels and processed using two stacked convolutional layers.

To enhance spatial locality, we arrange the stocks on the 2D grid using hierarchical clustering based on their correlation structure, ensuring that highly correlated stocks are placed near each other. The output of the CNN module captures spatial dependencies across the market and is subsequently fed into an LSTM for temporal sequence modeling.

Spatial Attention Since the correlation structure among stocks evolves over time, a static arrangement based on fixed correlation values may fail to capture such dynamic relationships accurately. To address this limitation, we introduce a spatial attention mechanism. Specifically, a 1×1 convolutional layer is added to dynamically generate an attention weight matrix, which is then fed into the subsequent convolutional layer for further processing.

The spatial attention layer guides the model to focus on the most informative regions of the input feature map—namely, groups of highly correlated stocks—while suppressing noise from low-correlation areas. This mechanism not only enhances predictive performance but also improves the overall stability of the model.

The attention weight matrix A is generated using a 1×1 convolution followed by a Sigmoid activation function:

$$A = \sigma(W_{\text{att}} * X^{(1)} + b_{\text{att}}) \quad (9)$$

- $W_{\text{att}} \in \mathbb{R}^{64 \times 1 \times 1 \times 1}$: the kernel parameters of the 1×1 convolution.
- $b_{\text{att}} \in \mathbb{R}$: the bias term.
- σ : the Sigmoid function, which maps values to the range $[0, 1]$.
- $A \in \mathbb{R}^{B \cdot T \times 1 \times H \times W}$: the resulting attention weight matrix.

The attention weights are applied to the feature map from the previous layer via element-wise multiplication:

$$X_{\text{att}} = X^{(1)} \odot A \quad (10)$$

where $\odot$ denotes element-wise multiplication.

The shape of the input to the next 2D convolutional layer remains unchanged, but the feature values at each spatial location are reweighted by the attention scores.

2.2 5-Minute Level

2.2.1 Data Description

- **Frequency & span:** 5-minute bars from **2023-10-02** through **2024-08-30**.
- **Universe:** 78 Taiwanese equities.
- **Features per stock:** 79 numeric columns — including current and lagged returns (`rtn`, `nextParkVol`), mid-price measures (`vwap`), order-book imbalance and spread ratios at multiple depths (`BP1`, `SP1`, `avgNormBV\*`, `avgNormSV\*`, etc.), futures-based analogues, a battery of technical indicators (price and volume momentum, RSI, MFI, Chaikin oscillators, Williams %R, SMA cross-rates, “park” volatility over various horizons), and more.

Cross-stock return correlation (5-min)

- **Diagonal = 1.0** (each stock perfectly correlated with itself).
- **Off-diagonals** overwhelmingly lie in $[-0.1, +0.1]$, indicating **very low pairwise correlation** on a 5-minute horizon.
- A handful of stock pairs exhibit slightly stronger co-movement (pockets around ± 0.2), but no large blocks of high correlation.

2.2.2 Feature Engineering

We implemented comprehensive data preprocessing to ensure feature quality:

- **Data Filtering:** Retained only numerical features, excluding temporal and categorical variables
- **Anomaly Handling:** Replaced infinite values ($\pm$) with medians and treated extreme values using 4 clipping
- **Missing Values:** Filled NaN entries with feature medians
- **Scaling:** Applied RobustScaler transformation to enhance model resilience

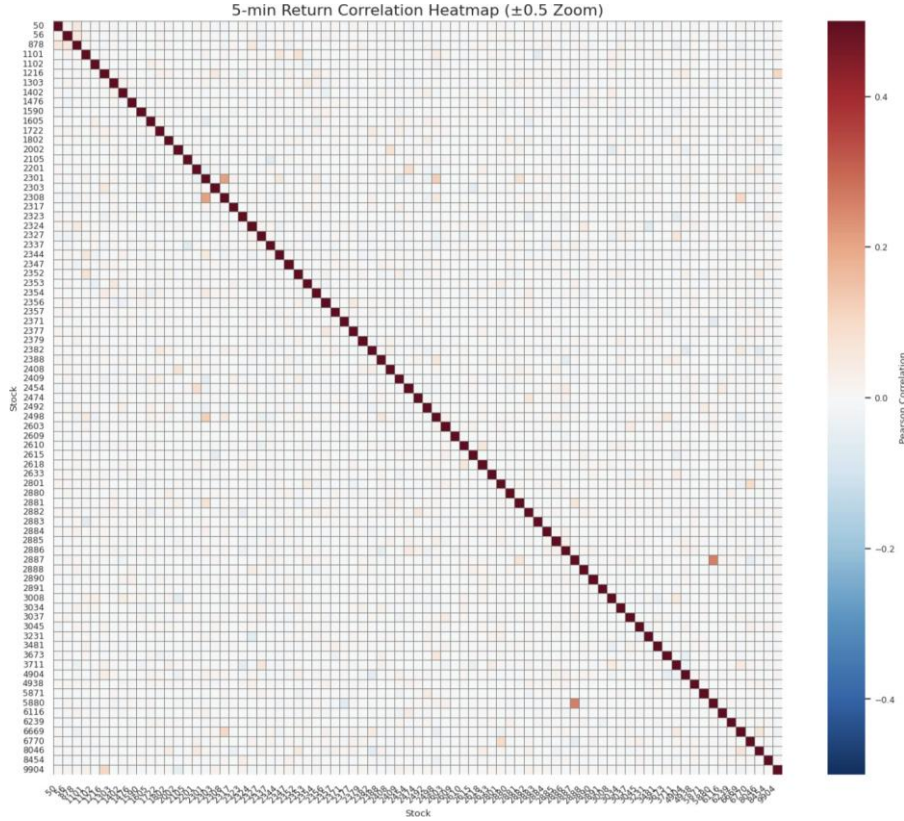


Figure 1: Correlation Matrix of stock pairs return on 5-min Level

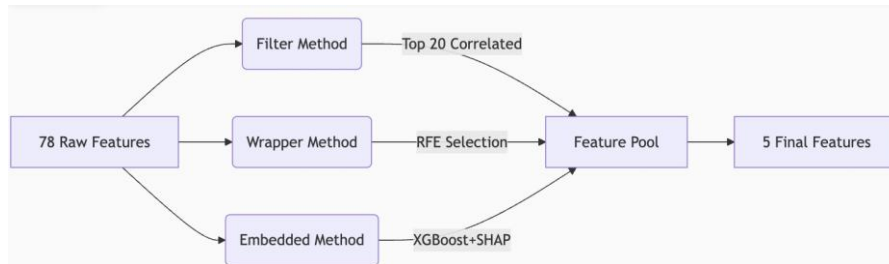


Figure 2: Feature Engineering Diagram

Post-processing verification confirmed zero infinite/missing values and all features within valid numerical ranges. Our three-stage selection methodology (Figure 2) integrates complementary approaches:

- Filter Stage: Identified top 20 features by absolute correlation with 30-minute forward returns
- Wrapper Stage: Recursive Feature Elimination (RFE) with dynamic step size (10% of features)
- Embedded Stage: Combined XGBoost importance and SHAP values into unified metric: $I_{\text{combined}} = \frac{1}{2}(I_{\text{XGBoost}} + I_{\text{SHAP}})$

Five high-predictive features met stringent selection criteria:

ParkVol5min AvgSlopeab ParkVol10min ParkVol15min AvgNormbv3

2.2.3 Model Architecture: Hybrid CNN-LSTM Network

We designed a specialized neural architecture integrating convolutional and recurrent networks for high-frequency financial forecasting. The model consists of three core components:

1. Feature Extraction Module

- Multi-scale convolutional pathways capture patterns at different temporal resolutions
- Hierarchical processing with exponentially expanding receptive fields
- Channel attention dynamically weights feature importance

2. Temporal Modeling Module

- Bidirectional LSTM layers model contextual dependencies
- Gating mechanisms filter noise while preserving signals

3. Prediction Module

- Feature dropout ensures robust output
- Linear transformation forecasts cumulative returns

Key Architectural Innovations

1. Multi-resolution Processing

Simultaneous extraction of short, medium, and long-term market patterns through parallel convolutional pathways.

2. Attention Mechanisms

Feature recalibration via squeeze-and-excitation blocks that adaptively weight financial factors:

$$\mathbf{y} = \sigma(\mathbf{W}_2 \delta(\mathbf{W}_1 \mathbf{z})) \odot \mathbf{x} \quad (11)$$

- $\mathbf{x}$: Input feature tensor
- $\mathbf{z} = \frac{1}{T} \sum_{t=1}^T \mathbf{x}_t$: Global context vector
- δ : ReLU activation function
- σ : Sigmoid activation
- $\mathbf{W}_1, \mathbf{W}_2$: Learned projection matrices
- $\odot$: Element-wise multiplication

3. Noise-robust Design

Hierarchical noise filtering through:

- Input-level Gaussian noise injection
- Multi-stage feature purification
- Regularization at multiple network levels



Figure 3: Architecture Diagram

2.2.4 Hyperparameter Settings

We implemented a rigorous Bayesian optimization framework to maximize model performance. The search strategy employed Tree-structured Parzen Estimators (TPE) across 32 configurations evaluated through temporal cross-validation. Key hyperparameters included architectural dimensions, regularization strengths, and sequence length. Optimization targeted the information coefficient (IC) while penalizing variance across validation folds. Optimal configurations emerged through iterative refinement:

- Sequence length: 78 timesteps
- Convolutional layers: 3 with attention reduction ratio 16:1
- Batch Size: 128
- Dropout Rate: 0.2
- Learning rate: 0.0001 with ReduceLROnPlateau scheduling

2.3 Minute Level

Due to the complexity of the experiment, only three months of minute-level data (October, November, and December, 2020) are used. The first 10% of the data is used for correlation calculation, signal-to-noise ratio (SNR) analysis, and pretraining of the LSTM model. The key characteristics are selected to predict the percentage change in VWAP, calculated as $\frac{\text{NextVwap} - \text{Vwap}}{\text{Vwap}} \times 100$. This methodology of the minute level aims to adapt correlation and ensemble techniques to achieve an optimal bias-variance trade-off, enhancing prediction accuracy in highly noisy data.

2.3.1 Correlation Analysis of minute level

The correlation is calculated based on the percentage change in Volume Weighted Average Price (VWAP) for each stock. Some stock pairs exhibit significant correlations, enabling their use to enhance LSTM predictions. However, as most pairs show weak correlations, only those with a correlation of 0.1 or higher are included to reduce computational complexity.

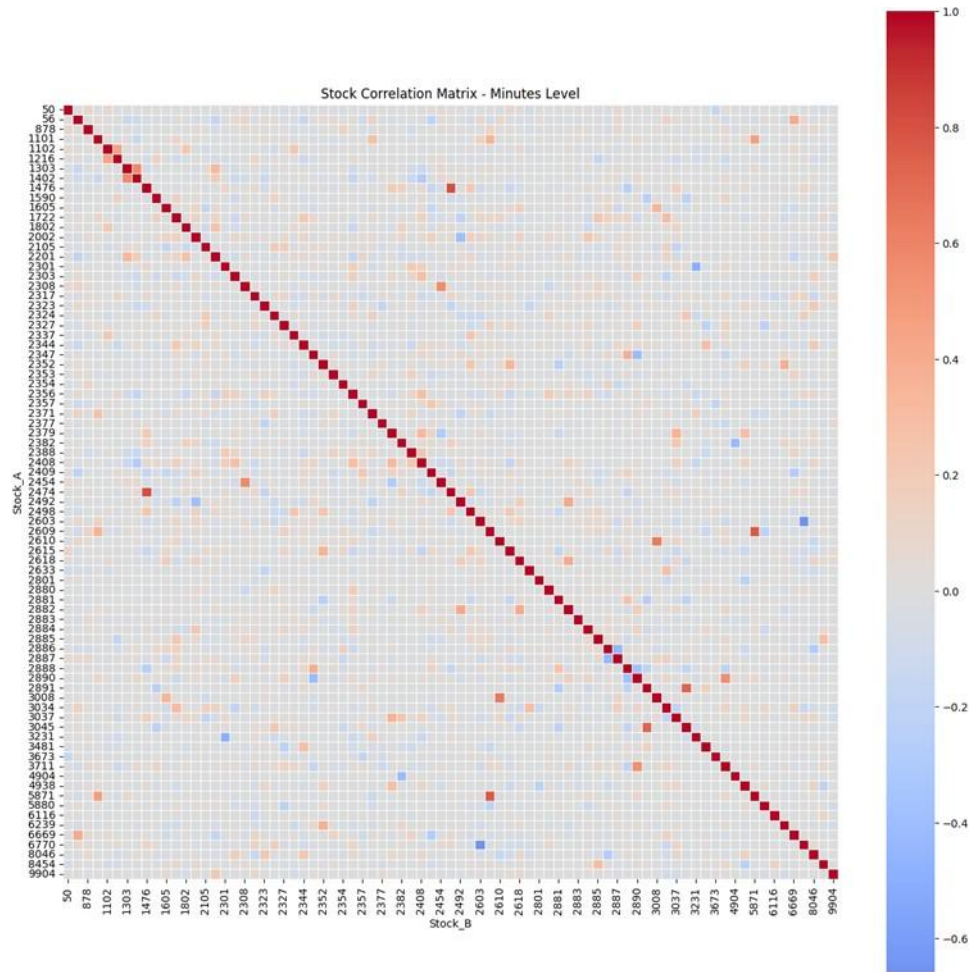


Figure 4: Correlation matrix of stock pairs based on percentage VWAP change for minute level.

2.3.2 Low Signal-to-Noise Ratio (SNR)

Minute-level stock data exhibits a low SNR, with minimal and noisy price changes. This leads models to fit noise rather than the actual signal, necessitating strategies to prevent overfitting. Linear Regression is employed to assess the SNR of features relative to the next VWAP.

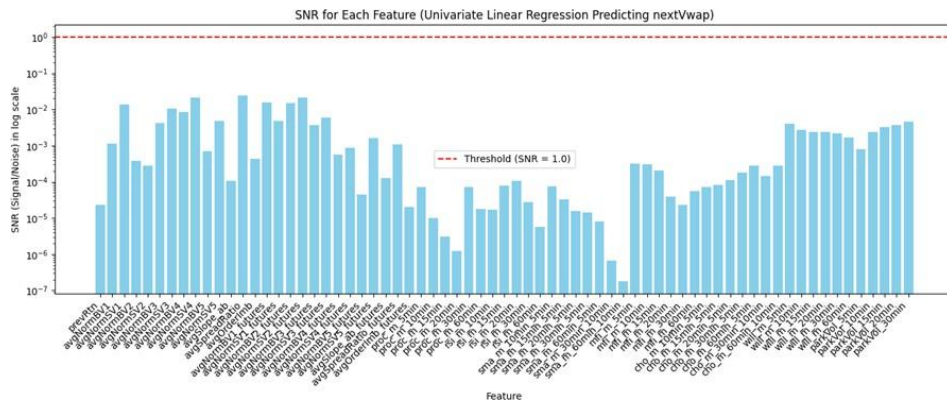


Figure 5: SNR analysis of features, indicating significant noise presence for minute level.

From the graph above, the features exhibit a low SNR, indicating a significant presence of noise.

$$\text{SNR} = \frac{R^2}{1 - R^2} \quad (12)$$

$$R^2 = 1 - \frac{SS_{\text{residuals}}}{SS_{\text{total}}} \quad (13)$$

To mitigate this, stock correlations are integrated into the prediction process, and an ensemble method is used to reduce model variance.

2.3.3 Correlation-Controlled LSTM Memory Flow

The memory cell of an LSTM preserves information over long sequences to capture and update past data. The correlation between stocks represents the degree of shared information or similarity, meaning that their historical data is assumed to contribute to each other's predictions. Each stock has its own dedicated LSTM model. The LSTM structure is modified to include a multiplication operator that multiplies the memory cell by the correlation value before passing it into the original LSTM structure. In other words, correlation acts as a weighting mechanism that controls how much information flows into the current prediction. If certain stocks are highly correlated with the stock being predicted, they receive a higher weighting, allowing more past information from their respective LSTMs to influence the current prediction.

$$\text{Memory Cell}'_{\text{stock}_i, \text{model}_j, \text{time}_t} = \text{Memory Cell}_{\text{stock}_i, \text{model}_j, \text{time}_t} \times \text{Correlation}_{\text{stock}_i, \text{stock}_j} \quad (14)$$

2.3.4 Ensemble Model

For stock pairs with a correlation of 0.1 or higher, their respective LSTM models will also be selected to contribute to the prediction of the current stock. The outputs from the selected LSTM models will be weighted according to their normalized correlation values. Since a stock's correlation with itself is always 1, its own LSTM model will retain the highest voting power in the prediction process. However, other correlated stocks will also have a certain level of influence. As a result, the predicted value of the current stock will remain closely aligned with its own LSTM model's prediction while being adjusted based on the contributions of significantly correlated stock models.

$$\text{Ensemble Prediction}_{\text{stock}_i} = \sum_j \text{Prediction}_{\text{stock}_j} \times \text{Correlation}_{\text{stock}_i, \text{stock}_j} \quad (15)$$

2.3.5 Bias and Variance Tradeoff

The ensemble method has been proven to improve model performance by introducing bias in individual predictors and combining their outputs to reduce overall variance. The correlation memory control adds randomness to long-term information, increasing bias except the LSTM of the current stock being predicted since the weighting is 1. Meanwhile, the predictions from each LSTM model are ensembled to further minimize variance. As a result, each LSTM model is less prone to overfitting in the noisy stock data environment, while the collective intelligence of multiple models helps to stabilize predictions.

2.3.6 Training and Testing Framework

During the training and testing process, each stock’s features and correlated pairs—those with a correlation of 0.1 or higher are fitted into LSTM models of different stocks. Within each LSTM model, correlation values from the corresponding stock pairs are used to control memory flow before passing the any gates and generating predictions for the current stock. All outputs are then combined to form the ensemble prediction, with individual predictions weighted according to their correlation values. Model performance is evaluated using Mean Squared Error and Information Coefficient. However, optimization relies solely on the stock’s own prediction output, rather than the ensemble prediction, for loss calculation and backpropagation.

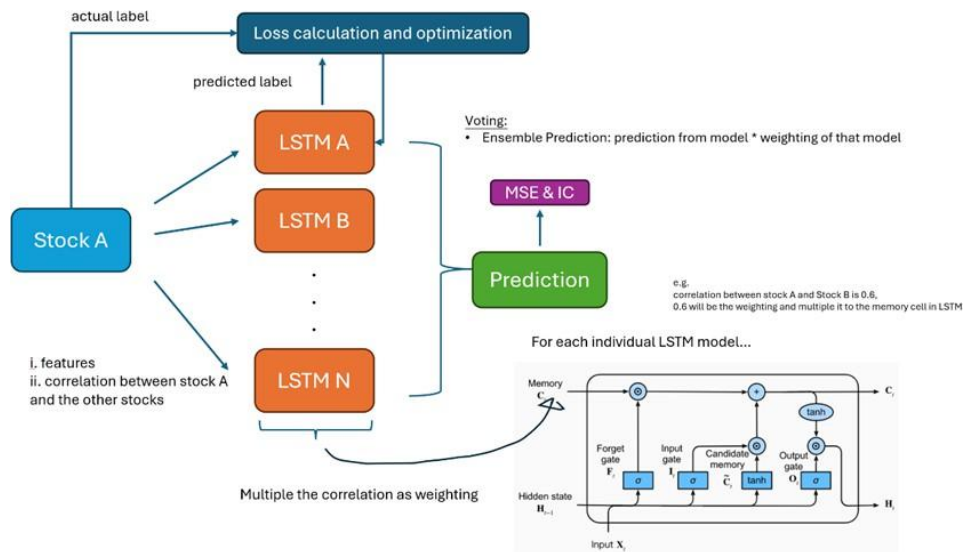


Figure 6: Training and testing process for the Correlation-adaptive Ensemble LSTM for minute level.

From a dimensional perspective, stock information is compressed from (number of stocks $\times$ number of features) to (number of stocks $\times$ 1), and finally to a single prediction (1 $\times$ 1).

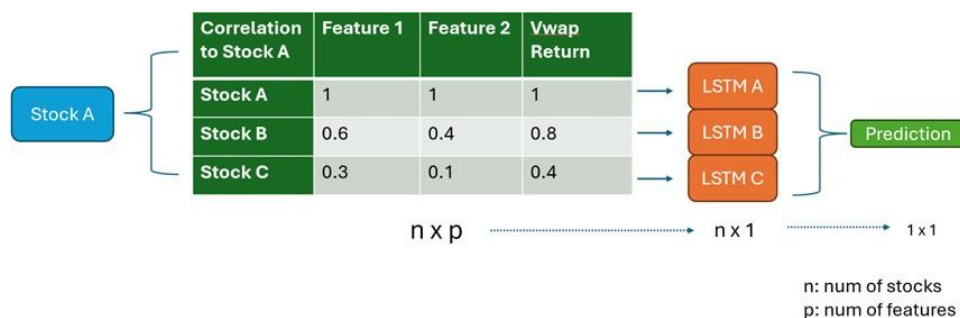


Figure 7: Layer-by-layer framework of the Correlation-adaptive Ensemble LSTM for minute level.

2.3.7 Hyperparameter Settings

After the first 10% of data is used for pretraining the LSTM model, rolling training and testing windows are applied, with a training window of 2 days and a testing window of 1 day. The

model is configured with a hidden size of 10, a sequence length of 50, a learning rate of 0.01, and 10 epochs, using the Adam optimizer. The purpose of this setup is to minimize the risk of overfitting by employing shorter window sizes, a smaller hidden size, a lower learning rate, and fewer epochs, thereby preventing large updates to the model.

2.3.8 Feature Importance

To simplify the data and enhance model efficiency, the importance of each feature will be assessed using a Random Forest model. Only features with a significance score of 0.05 or higher will be selected for training. As a result, 38 features are chosen out of the total 71, which is a large decrease in terms of data complexity.

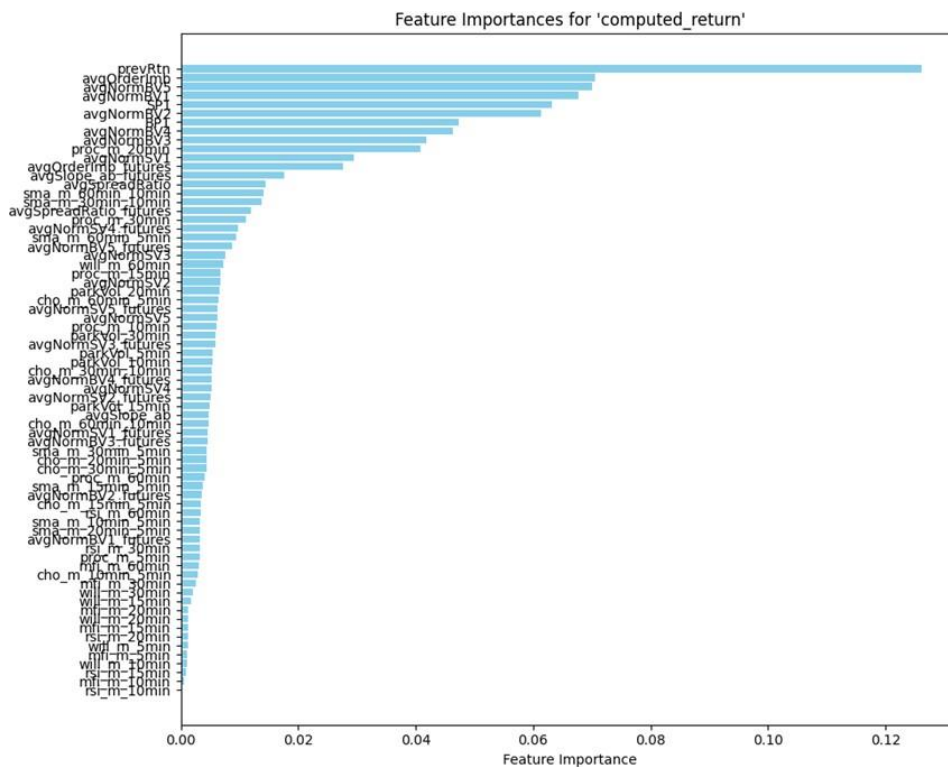


Figure 8: Feature importance scores from Random Forest analysis for minute level.

3 Results

3.1 Monthly Level

3.2 Simple LSTM

For data with monthly frequency, we trained the LSTM model using a regression approach. A separate LSTM model was built for each individual stock. The dataset was split into 80% for training and 20% for testing. The model was trained on the training set and evaluated on the testing set, with Mean Squared Error (MSE) computed on the test set and averaged across stocks.

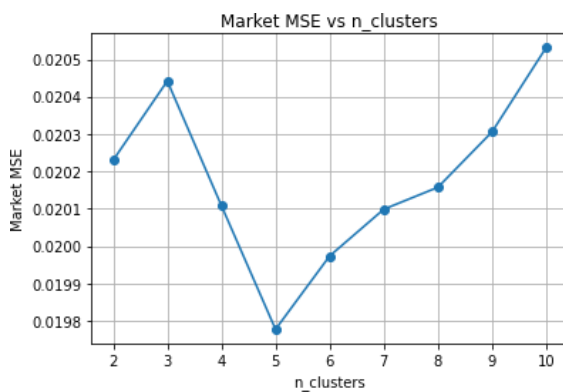
The time window used was 30 months. The learning rate was set to 0.001, and the number of

training epochs was 5. For the simple LSTM baseline, the average MSE was 0.026269 and the Information Coefficient (IC) was 0.042074.

Change the structure of the input data:

3.2.1 Results for LSTM with Clustering

We applied KMeans clustering based on historical stock return data and implemented weight inheritance within each cluster. This study investigates how different values of the number of clusters k affect the performance of the LSTM model. As shown in the figure??, the MSE values after clustering are generally lower than those of the simple LSTM without clustering.



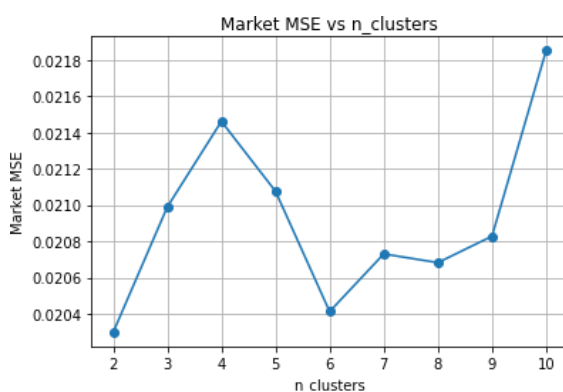
(a) MSE of LSTM with clustering (KMeans)



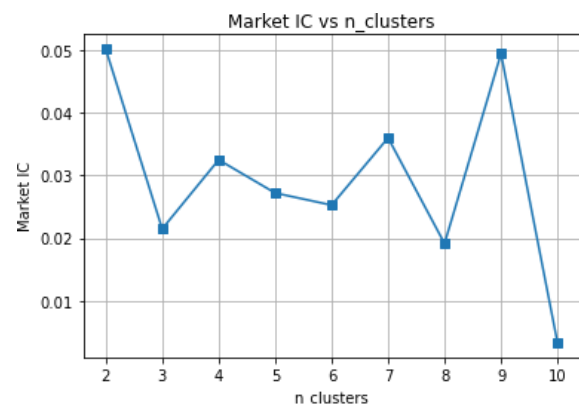
(b) IC of LSTM with clustering (KMeans)

Figure 9: Performance of LSTM with clustering (KMeans): MSE and IC under different numbers of clusters k .

We performed hierarchical clustering based on pairwise stock correlations and applied weight inheritance within each cluster. The MSE and IC values for different numbers of clusters k are shown in the figure10. Similar to KMeans clustering, the MSE values from hierarchical clustering are generally lower than those of the simple LSTM model. Notably, when $k = 9$, the IC reaches a relatively high value of 0.494.



(a) MSE of LSTM with clustering



(b) IC of LSTM with clustering

Figure 10: Results of LSTM with clustering

Combining the results from both clustering methods, we observe that hierarchical clustering based on stock correlation achieves the best prediction performance when $k = 9$. These results suggest that incorporating inter-stock correlation can significantly improve the prediction accuracy of stock returns.

3.2.2 Results for LSTM with Feature Concatenation

The parameter settings are consistent with those of the simple LSTM. The training and testing split remains at 80% and 20%, respectively. Experimental results show that concatenating features from $k = 9$ neighboring stocks yields the best performance, with an MSE of 0.027121 and an IC of 0.012134. However, compared to the simple LSTM, the improvement in MSE is marginal, and the IC shows a noticeable decline.

After feature concatenation, the number of feature columns increases significantly, exceeding 500 in some cases. This substantial increase in dimensionality introduces a higher risk of overfitting, especially given the relatively small size of the training set. As a result, the model may capture noise rather than true underlying patterns, leading to reduced generalization performance on the test set. This could explain why the concatenation-based LSTM underperforms the simple LSTM in terms of overall predictive accuracy, particularly as reflected in the lower IC values.

Change the structure of the model:

3.2.3 Results for LSTM with Correlation

Since the internal structure of the LSTM was modified to incorporate cross-stock correlation—specifically by introducing contextual information into the input gate—the model was implemented using a custom-built LSTM architecture rather than relying on standard library functions. The dataset split remained consistent with previous experiments, with 80% used for training and 20% for testing.

On the test set, the model achieved a Mean Squared Error (MSE) of 0.012887. However, the Information Coefficient (IC) was -0.075514, indicating a negative correlation between predicted and actual returns. In absolute terms, the IC value is 0.075514.

The proposed model is less prone to overfitting due to its use of hidden states as a compressed and abstract representation of the historical information in the input sequence. Instead of directly concatenating raw features from other stocks—which would significantly increase the input dimensionality and risk overfitting—the model integrates information from correlated stocks via their hidden states. These hidden states serve as abstract features that carry distilled temporal signals, effectively incorporating market-wide information without introducing excessive complexity.

Moreover, since each stock’s training sample indirectly includes contextual information from related stocks, the model benefits from an expanded set of effective training signals. This implicit data augmentation helps reduce overfitting risk by encouraging the model to learn generalizable patterns rather than stock-specific noise.

3.2.4 Results for CNN-LSTM

Market Cube The kernel size is set to $3 \times 3 \times 55$, and the number of kernels is 64. Stock data is arranged according to hierarchical clustering based on correlation before being fed into the CNN.

In the first 2D convolutional layer (CNN2D), each kernel is a three-dimensional tensor of size $3 \times 3 \times 55$, consisting of 55 separate 3×3 weight matrices—each corresponding to one feature channel. Taking kernel 1 as an example, the computation proceeds as follows: for feature 1, the 9 feature values from a 3×3 patch of 9 stocks are multiplied element-wise with the corresponding 3×3 weight matrix in kernel 1 (channel 1), resulting in a single intermediate value (i.e., the contribution from channel 1). This process is repeated across all 55 channels, generating 55 intermediate contributions. These are then summed to produce the final output value at a specific position in the feature map. A total of 64 such values are generated—one for each kernel—producing 64 output channels. The resulting output feature map is then passed to the second CNN2D layer, where a similar convolutional process is applied.

The MarketCube model achieved a Mean Squared Error (MSE) of 0.020948 and an Information Coefficient (IC) of 0.026543. While the MSE is lower than that of the simple LSTM model, indicating improved error performance, the IC remains relatively modest.

Spatial Attention To improve the IC value while maintaining a low MSE, we introduced a direction-aware penalty term into the loss function. This modification encourages the model not only to minimize prediction error but also to align more closely with the correct return direction.

Additionally, a spatial attention mechanism was integrated into the MarketCube framework. Specifically, a new convolutional layer was inserted between the two CNN2D layers to dynamically generate an attention weight matrix. This attention layer reweights the output feature map from the first CNN2D layer before passing it into the second CNN2D layer, guiding the model to focus on more informative regions.

With these enhancements, the model achieved an MSE of 0.019609 and an IC of 0.079450. Compared to the benchmark, these results demonstrate that incorporating spatial attention and directional penalties significantly improves both prediction accuracy and directional consistency.

Table 1: Performance Comparison of Models

Models	MSE	IC
Simple LSTM	0.026269	0.042074
LSTM with Clustering	0.0208	0.494
LSTM with Feature Concatenation	0.027121	0.012134
LSTM with Correlation (Custom Gate)	0.012887	0.075514
CNN-LSTM: Market Cube	0.020948	0.026543
CNN-LSTM: Spatial Attention	0.019609	0.079450

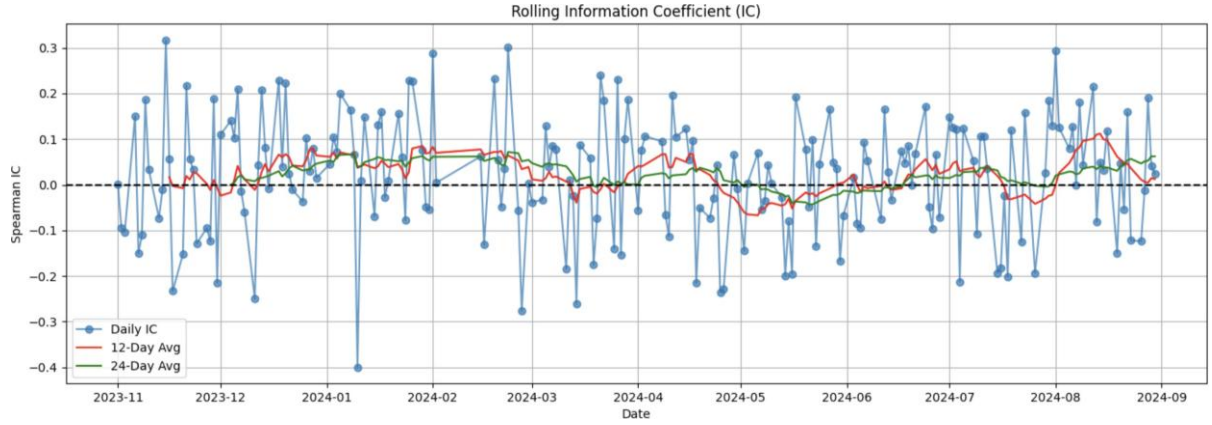


Figure 11: Rolling Shared-Weight Model Rolling IC

The table 1 presents the results of all models, where the reported performance for both LSTM with Clustering and LSTM with Feature Concatenation corresponds to their best-performing configurations.

3.3 5-Minute Level

3.3.1 Experiment Overview

This section compares five modeling strategies on Taiwan 78-stock 5-minute bars (2023-10-02 to 2024-08-30):

1. Baseline LSTM with weight inheritance
2. Independent LSTM (no weight inheritance)
3. Shared-weight model (one-time training over all stocks)
4. Rolling shared-weight model (retrain each day)
5. Cluster-based rolling shared-weight model

3.3.2 Performance Metrics

We evaluate each model by mean squared error (MSE) and Spearman information coefficient (IC). Lower MSE and higher IC indicate better point prediction and directional signal quality, respectively.

Table 2: Overall Mean MSE and Mean IC for 5-Minute Models

Model	Mean MSE	Mean IC
Baseline LSTM (weight inheritance)	0.01832	0.01643
Independent LSTM (no inheritance)	0.02453	0.00928
Shared-weight model (one-time training)	0.01640	0.01830
Rolling shared-weight model	0.00842	0.02090

3.3.3 Discussion

- **Inheritance vs. Independent:** Sharing weights across stocks reduces MSE by $\sim 33\%$ and roughly doubles IC, showing that market-wide features improve stability.
- **One-time vs. Rolling:** Online re-training each day further halves MSE (to 0.0084) and raises IC to 0.0209, confirming the value of discarding stale patterns.
- **Best Practice:** Combining parameter sharing with rolling updates achieves the best fit and directionality on high-frequency data.

3.3.4 Cluster-Based Rolling Training

We further group stocks by feature-correlation into clusters of various sizes and run the rolling shared-weight procedure within each cluster. Table 3 summarizes the cluster-level IC statistics.

Table 3: Cluster-Based Rolling IC Analysis

Cluster Size	Mean IC	IC Std. Dev.	Positive IC Ratio	IC > 0.05 Ratio
3	0.01821	0.13223	55.6%	43.5%
5	0.01930	0.14244	55.1%	41.1%
10	0.00956	0.13781	49.3%	34.3%
20	0.02542	0.14318	59.9%	45.4%
30	0.05662	0.13981	67.6%	53.6%
50	0.03010	0.19541	58.9%	49.3%

3.4 Minute Level

The following are the results within the testing windows.

3.4.1 MSE and IC Analysis

The ensemble model is compared to an independent model in which the prediction of each stock is based solely on its own LSTM.

Table 4: Mean Squared Error (MSE) Comparison for the minute level

Metric	Value
Average MSE of Ensemble Model	173.55
Average MSE of Independent Model	183.955
Average MSE Decrease by Ensemble Model	10.40

Table 5: Information Coefficient (IC) Comparison for the minute level

Metric	Value
Average IC of Ensemble Model	0.01500
Average IC of Independent Model	0.01031
Average IC Increase by Ensemble Model	0.00469

Based on both MSE and IC metrics, the correlation-adaptive ensemble method outperforms the independent method. This indicates that incorporating correlation and ensemble techniques leads to better stock predictions at the minute level.

3.4.2 Leading and Being Led Stocks

The top 20 leading stocks (highest average correlation) and bottom 20 being the led stocks are:

- **Leading Stocks:** 2002, 2882, 5871, 1101, 1722, 3231, 3711, 2881, 2618, 2474, 2609, 2408, 2610, 3037, 2498, 5880, 1476, 2890, 3008, 2352
- **Being Led Stocks:** 2615, 4938, 2353, 2301, 1605, 50, 2891, 2883, 8454, 6669, 2303, 2337, 2371, 3045, 2357, 2887, 2356, 6116, 1802, 8046

3.4.3 Top 10 Stocks with Highest MSE Decrease

	Stock	Independent Model MSE	Ensemble Model MSE	MSE_decrease by Ensemble Model
43	2609	9793.511774	8921.527615	871.984159
35	2388	136.094891	91.275918	44.818973
63	3045	78.139648	51.941039	26.198610
20	2323	151.292707	133.143187	18.149520
11	1722	422.385663	416.345749	6.039914
12	1802	141.422022	135.780599	5.641422
51	2882	52.711369	47.127262	5.584107
57	2888	193.671845	189.819061	3.852783
15	2201	155.647660	153.108766	2.538893
77	8454	199.422149	197.327666	2.094482

Figure 12: Top 10 stocks with the highest MSE decrease (leading stocks in red).

The above result shows there are more leading stocks (Red) to have higher MSE decrease than being led stocks, ensemble method gives a lower variance.

3.4.4 Top 20 Stocks with Highest IC Increase

	Stock	Independent Model IC	Ensemble Model IC	IC increase by Ensemble Model
26	2352	-0.055508	0.061303	0.116810
4	1102	-0.082152	0.019348	0.101500
50	2881	-0.086157	-0.003062	0.083095
21	2324	-0.059150	0.019014	0.078164
38	2454	-0.105189	-0.032230	0.072959
33	2379	0.079407	0.142075	0.062668
8	1476	-0.040337	0.019091	0.059428
37	2409	-0.001695	0.056336	0.058031
63	3045	0.007661	0.061705	0.054044
59	2891	-0.069669	-0.017319	0.052351

Figure 13: Top 20 stocks with the highest IC increase (leading stocks in red).

The above result shows there are more leading stocks (Red) to have higher IC increase than being leaded stock, ensemble method gives a better accuracy.

The results indicate that the predictions of leading stocks’ models are adjusted based on other stocks’ model predictions, leading to greater accuracy. Therefore, correlation memory control and ensemble methods significantly enhance the performance of leading stocks.

3.4.5 Complexity Analysis

Training and testing durations for 3 months of minute-level data (80 stocks) are:

Table 6: Training and Testing Duration for the minute level

Method	Duration
Independent Method	75 hours (1.19h/day)
Ensemble Method	93 hours (1.48h/day)

The hyperparameter settings ensure practical feasibility for real-world applications.

4 Discussion

4.1 Monthly Level

Simplified MarketCube Layout The current MarketCube model relies on a one-time hierarchical clustering to arrange stocks spatially. However, this static arrangement may not fully reflect the dynamic nature of stock correlations. A more flexible or adaptive layout strategy could potentially enhance the CNN’s ability to capture spatial patterns.

Risk of overfitting due to model complexity With the introduction of multiple model components (e.g., CNN layers, attention mechanisms, custom gates), the overall model complexity increases, which may lead to overfitting. Additional regularization techniques or more robust validation strategies are needed to ensure generalizability.

4.2 5-Minute Level

The 5-minute horizon presented a uniquely challenging environment: pairwise stock return correlations were almost entirely confined to $[-0.1, +0.1]$, indicating that individual stock returns evolve largely independently on this time scale. Despite this low cross-sectional signal, our experiments demonstrated that *shared-weight* models and *rolling* updates can still extract meaningful common patterns and improve both point-prediction accuracy and directional consistency.

- **Weight Inheritance vs. Independent Training.** Sharing a single CNN–LSTM across all 78 stocks (with sequential fine-tuning via weight inheritance) reduced average MSE from 0.02453 to 0.01832, and doubled the mean IC from 0.00928 to 0.01643. This confirms that even weak inter-stock dependencies carry useful information for denoising high-frequency returns.
- **One-Time vs. Rolling Re-Training.** Retraining the shared model at each trading day (“rolling” scheme) halved the MSE again to 0.00842 and boosted mean IC to 0.02090.

Discarding stale patterns and adapting daily to the evolving microstructure proved critical on such a noisy time scale.

- **Cluster-Based Rolling Updates.** Grouping stocks into clusters based on feature correlation before rolling training yielded further IC gains for certain cluster sizes. In particular, clusters of size 30 achieved the highest average IC (0.05662), with 67.6 of days showing positive IC and 53.6 exceeding 0.05. This suggests that modestly sized, correlation-homogeneous groupings strike a balance between model capacity and overfitting risk.

Taken together, these findings indicate that *parameter sharing* combined with *daily re-training* and *cluster-aware grouping* can overcome the inherently low signal-to-noise ratio of 5-minute returns. Even when pairwise correlations are very small, structured information across stocks accumulates to produce nontrivial forecasting improvements.

4.3 Minute Level

Overfitting should be a top priority, especially when handling high-frequency data. In addition to utilizing correlation and ensemble methods, further approaches can be explored. The current method considers correlation only in terms of percentage VWAP change between stocks; however, feature correlation can also be leveraged. A more informative correlation value can be derived from the average correlation between features of different stocks or by weighting them according to feature importance. Currently, the correlation value is obtained from only 10% of the data. However, if computational power allows, it could be incorporated within the rolling training and testing windows, as this approach may better capture the dynamic correlation properties over time.

5 Conclusion

5.1 Monthly Level

In summary, these results suggest that incorporating inter-stock correlation into LSTM-based models can effectively improve the accuracy of stock return prediction.

5.2 5-Minute Level

In the high-frequency (5-minute) setting, our experiments demonstrate that:

1. **Shared-Weight Architectures** significantly outperform independent models, reducing average MSE by over 30 and doubling directional IC.
2. **Rolling Re-Training**—updating the model daily—further cuts MSE in half and raises mean IC by 27 %, underscoring the importance of discarding outdated parameters in rapidly changing market conditions.
3. **Cluster-Based Training** exploits residual cross-sectional structure: clusters of size around 30 stocks yielded the best average IC (0.0566), suggesting an optimal granularity for group-wise parameter sharing.

Overall, our 5-minute-level study confirms that even in an almost uncorrelated market, *Correlation-Adaptive CNN-LSTM* frameworks—when augmented by rolling updates and cluster-aware

schemes—can extract enough common microstructure patterns to deliver materially better forecasting performance. Future work may explore dynamic, time-varying clustering and adaptive learning-rate schedules to further stabilize predictions in high-noise environments.”

5.3 Minute Level

The Correlation Memory Control Ensemble LSTM model effectively improves stock price prediction accuracy by leveraging stock correlations and ensemble techniques. The approach reduces MSE and increases IC compared to independent models, particularly benefiting leading stocks. Future work could explore dynamic correlation updates and feature-based correlations to further enhance performance.

6 References

References

- [1] Wu Yuheng, Zhang Xiaochang, and An Tianyuan. *Application of LSTM in Multi-Alpha Stock Factors: A Case Study of Taiwan Stock Market*. Stock Market Research Journal, 2025.